TDA352/DIT352 CRYPTOGRAPHY

Module 1 : Lecture 1 - Agenda

Introduction

- Cryptography: Meaning and Aims
- Brief History
- Kerckhoffs's Principle
- Foundations of Modern Cryptography

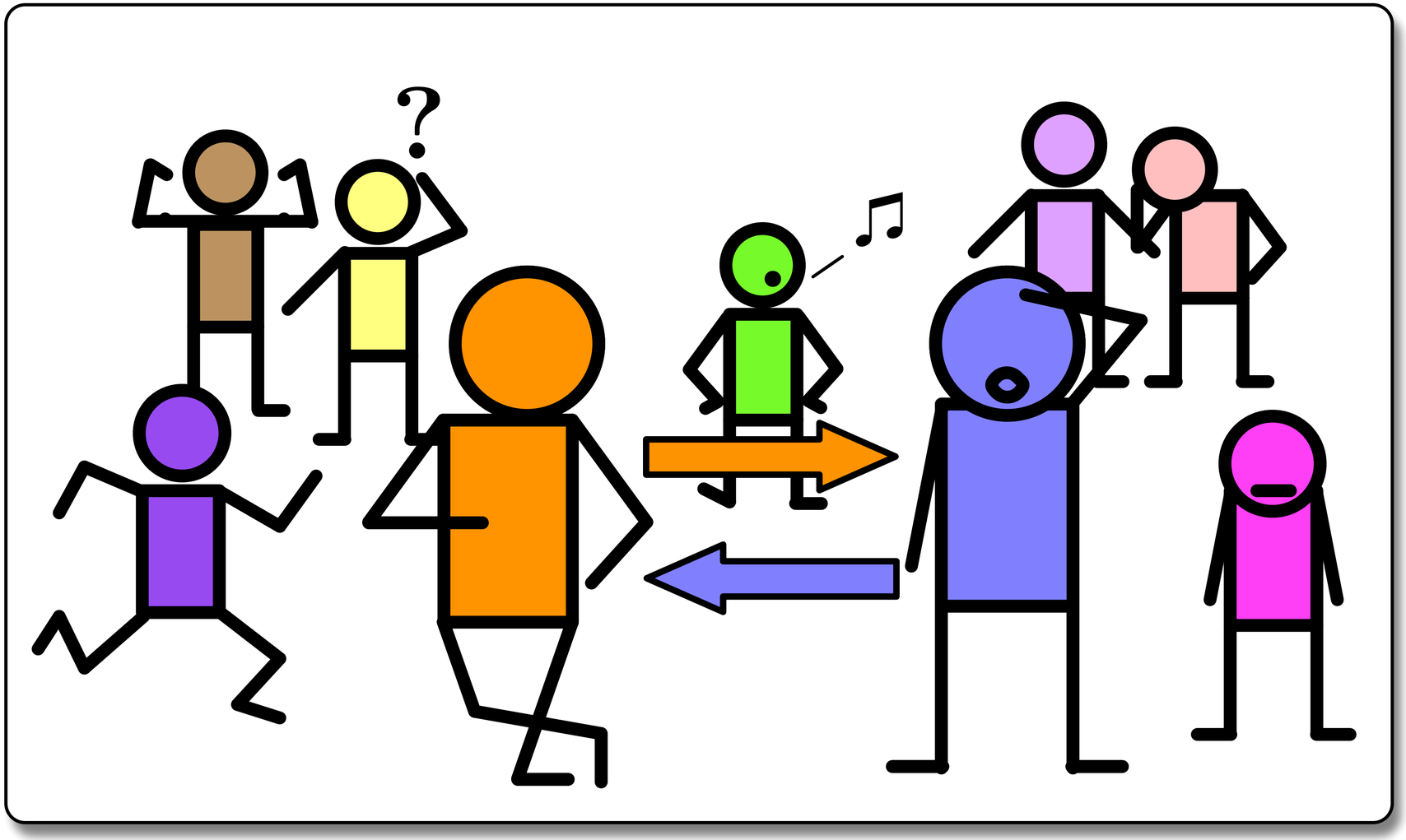
Symmetric Encryption

- Secure Communication
- Symmetric Encryption
- The One Time Pad (OTP) **[Proof]**
- Perfect Secrecy
- Shannon's Theorem **[Proof]**

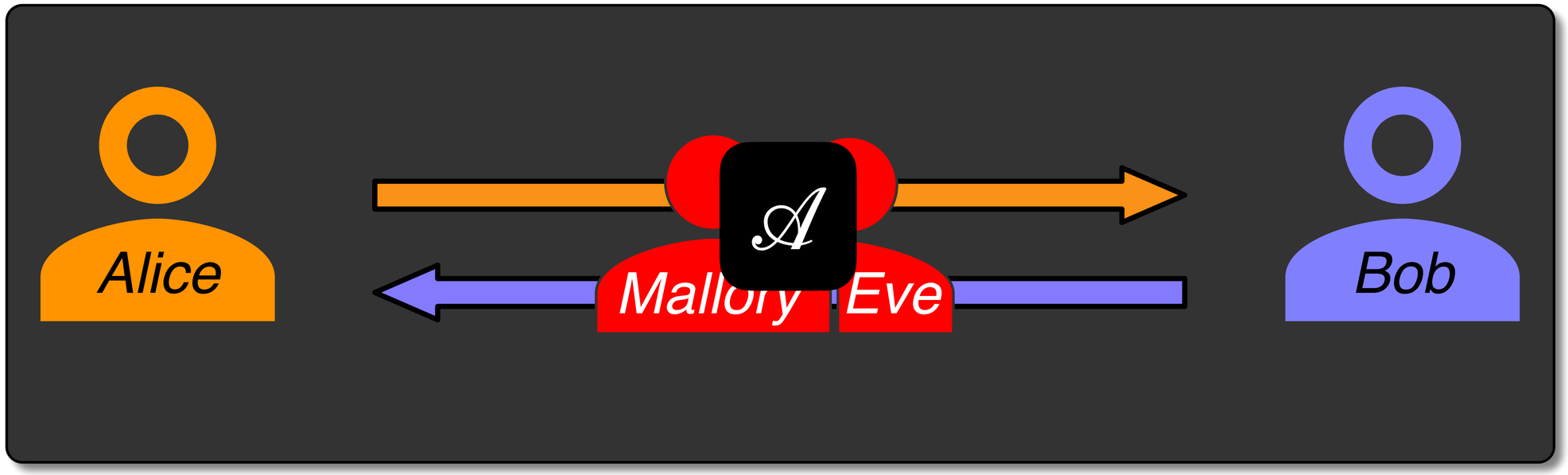
Pseudorandom Generators (PRG)

- Definition
- Examples

The Real World



The Dark World of Cryptography (and Cryptographers)
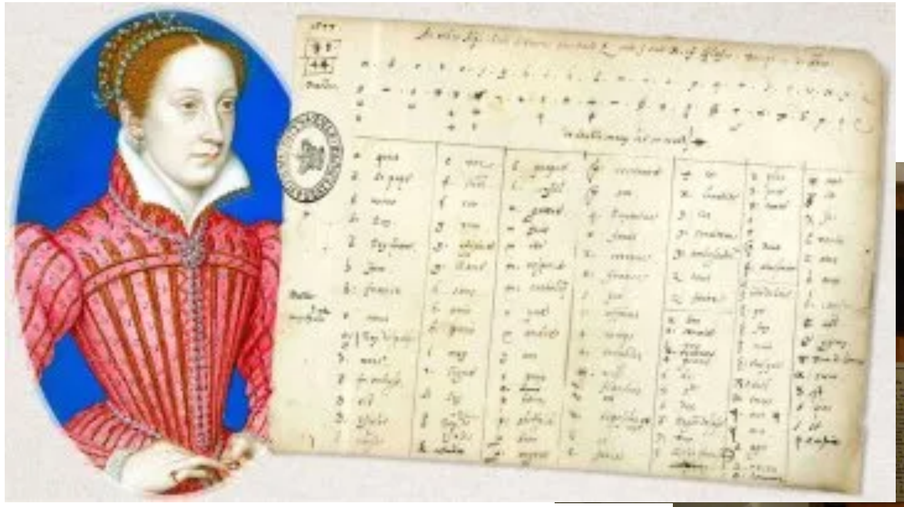


The Goal of Cryptography: “Make our Digital World Safe”

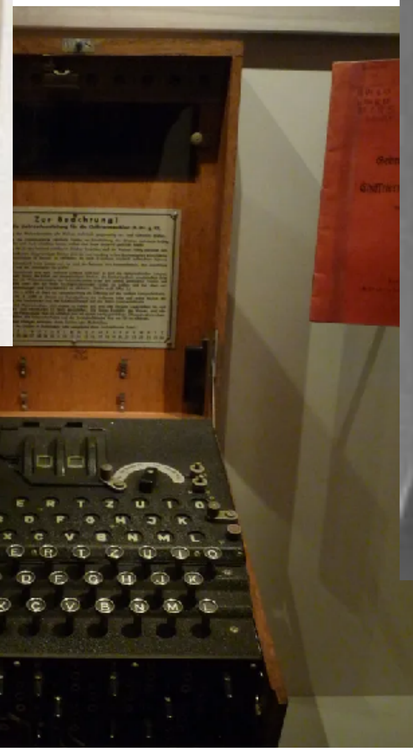
- 🔒 **Confidentiality**
- 💌 **Data integrity**
- 🖋️ **Authenticity**
- 🪪 **Entity identification**
- ⛔ **Access control / authorisation**
- 🤡 **Anonymity**
- ✅ **Non-repudiation**
- 👁️ **Privacy**

Brief History





Mary Queen of Scots



The Enigma Machine (Mavis Batey)



Gene Grabeel



Gaius Julius Caesar

Who?

Common issues:

- No rigorous security estimates / analysis
- “Security by Obscurity”
- Bad security “hygiene”

Kerckhoffs's Principle

Kerckhoffs's Principle (1883)
A cryptographic system should be secure even if everything about the system, except for the key, is public knowledge.

In Claude Shannon's words: **the enemy knows the system**, so we should design systems under the assumption that the enemy will immediately gain full familiarity with them (no security via obscurity)

Foundations of *Modern* Cryptography (1980-Now)

CRYPTOGRAPHY

CRYPTANALYSIS

CRYPTOLOGY

Rigorous definitions

- What does security mean?
- What are the attacker's goal and resources
- Precise mathematical security assumptions

(formally define "hard")

Rigorous logical reasoning to prove security

Lots of heuristics to define exact security levels
Solutions need to work in practice

- Efficient algorithms
- Use the lowest ratio $\frac{\text{length}}{\text{security}}$

When I say "crypto" I mean "cryptography" not "cryptocurrency"

TDA352/DIT352 CRYPTOGRAPHY

Module 1 : Lecture 1 - Agenda

Introduction

- Cryptography: Meaning and Aims
- Brief History
- Kerckhoffs's Principle
- Foundations of Modern Cryptography

Symmetric Encryption

- Secure Communication
- Symmetric Encryption
- The One Time Pad (OTP) **[Proof]**
- Perfect Secrecy
- Shannon's Theorem **[Proof]**

Pseudorandom Generators (PRG)

- Definition
- Examples

Useful Terminology

Efficient Algorithm : an algorithm that runs in time polynomial in its input (fast).

Scheme : a collection of algorithms that together satisfy a number of properties.

Protocol : a scheme where algorithms usually interact with one another.

Security parameter : $Alg(1^n)$ an n-bit string of only ones.

Conditional Probability : $Pr[A \mid B]$ (probability of event A occurring given event B.

More Terminology and Notation

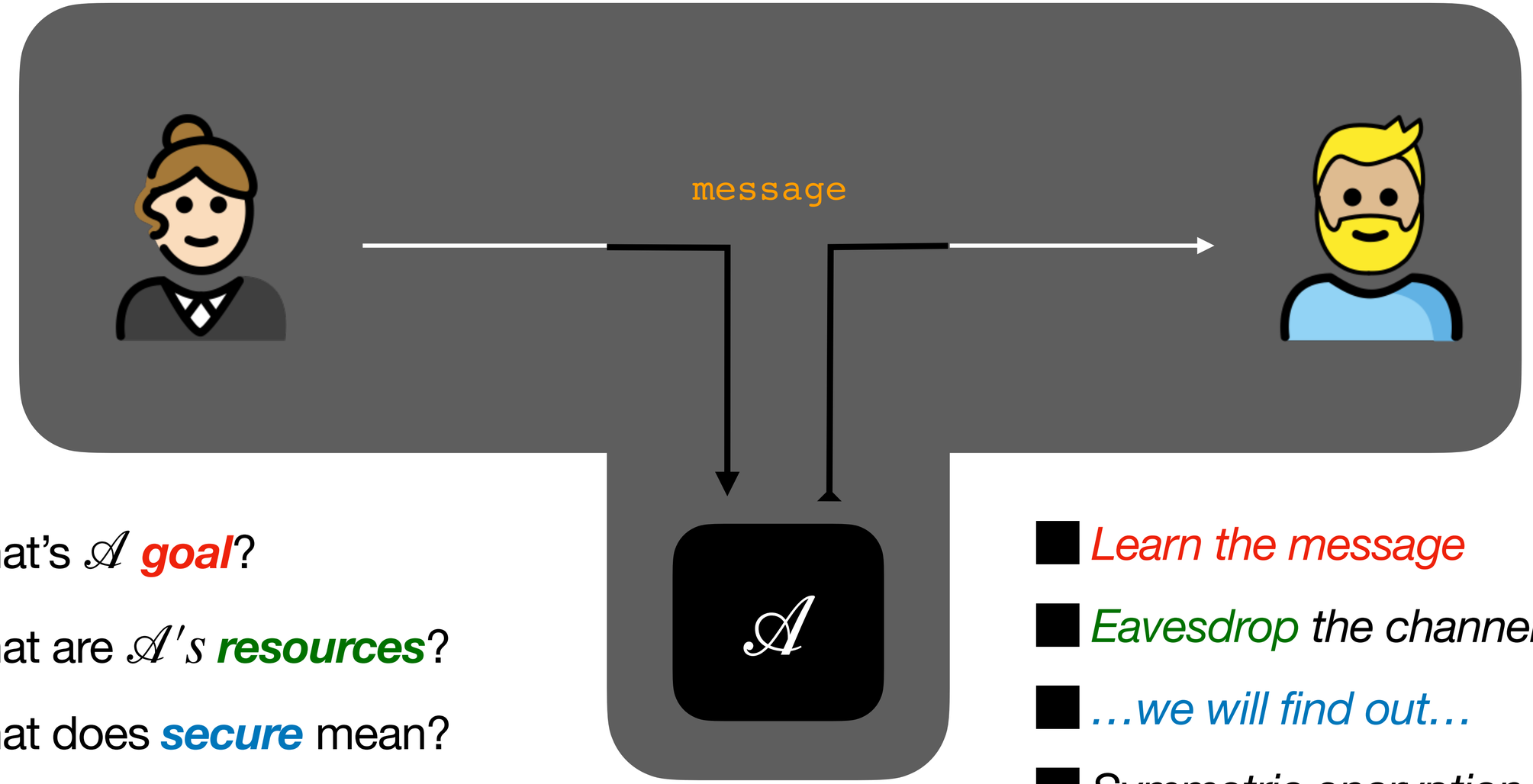
Deterministic : refers to a value that is set, or to a function that given an input always returns the same output. Notation: $b = 0$, $Alg(x) = y$

Random : refers to a value that is drawn from a set using the uniform distribution (all possibilities are equiprobable). Notation: $b \leftarrow \$\{0,1\}$

Randomised or **Probabilistic** : refers to a function or algorithm that involves sampling and using randomness, thus the output is non-deterministic (unless the randomness is specified).

Notation: $y \leftarrow Alg(x)$ and there exists $rnd \in \{0,1\}^n$ such that $y = Alg(x; rnd)$

Secure Communication Over an Insecure Channel



- What's $\mathscr{A}$ ***goal***?
- What are $\mathscr{A}$*'s* ***resources***?
- What does ***secure*** mean?
- What can Alice and Bob **use**?

- *Learn the message*
- *Eavesdrop the channel*
- *…we will find out…*
- *Symmetric encryption*

Symmetric Encryption - Syntax

Definition: Symmetric Encryption

A tuple $(KeyGen, E, D)$ is a symmetric encryption scheme over the sets $\mathscr{K}$ (key space), $\mathscr{M}$ (message space), and $\mathscr{C}$ (ciphertext space) if all algorithms are efficient and satisfy the following:

$KeyGen(1^n) \rightarrow k$: the key generation is a randomised algorithm that returns a key k. (This algorithm is often implicit when $k \leftarrow \$\mathscr{K}$)

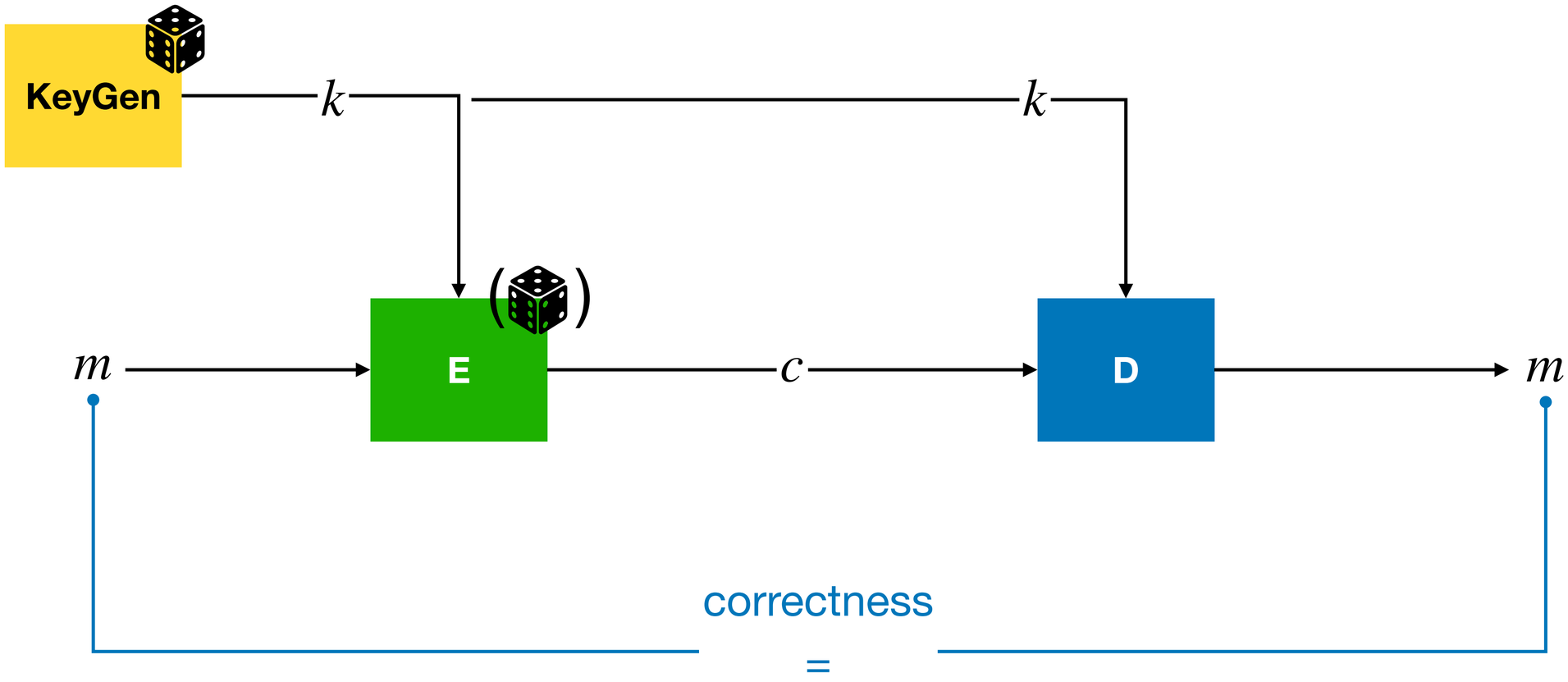
$E(k, m) \rightarrow c$: the encryption is a possibly randomised algorithm that on input a key k and a (plaintext) message m, outputs a ciphertext c.

$D(k, c) \rightarrow m$: the decryption is a deterministic algorithm that on input a key k and ciphertext c, outputs a plaintext message m.

CORRECTNESS:

$Pr[\ D(k, E(k, m)) = m\ \ | k \leftarrow KeyGen(1^n)] = 1$ …. for all messages $m \in \mathscr{M}$

Symmetric Encryption - Visualisation



Symmetric Encryption - the One Time Pad (OTP)

Definition: Symmetric Encryption

A tuple $(KeyGen, E, D)$ is a symmet
$\mathcal{K}$ (key space), $\mathcal{M}$ (message spac
algorithms are efficient and satisfy the

$KeyGen(1^n) \rightarrow k$: the key generat
returns a key k. (This algorithm is often
$E(k, m) \rightarrow c$: the encryption is a po
input a key k and a (plaintext) message
$D(k, c) \rightarrow m$: the decryption is a de
key k and ciphertext c, outputs a plain

Example: the One Time Pad (OTP)

$\mathcal{K} = \mathcal{M} = \mathcal{C} = \{0,1\}^n$

$KeyGen(1^n) \rightarrow k \ (where \ k \leftarrow \$\{0,1\}^n)$

$E(k, m) = k \oplus m$

$D(k, c) = k \oplus c$ 🧐 *how to decrypt?*

Key:	1	0	1	1	0	0	1	1	1	0	0	1
	⊕											
Plaintext:	0	1	1	0	1	0	0	0	1	1	0	1
Ciphertext:	1	1	0	1	1	0	1	1	0	1	0	0

CORRECTNESS:

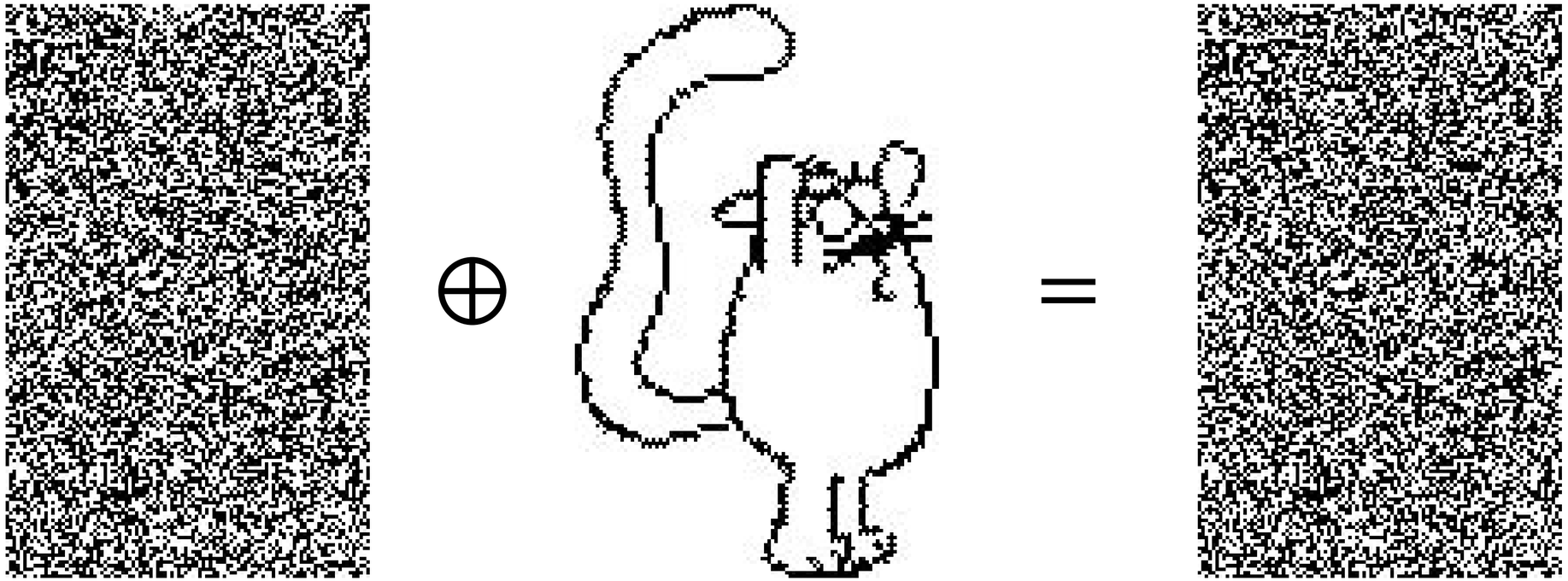
$Pr[\ D(k, E(k, m)) = m \ \ | k \leftarrow KeyGen(1^n)] = 1$ …. for all messages $m \in \mathcal{M}$

OTP From the Attacker's Point of View

Uniformly random key

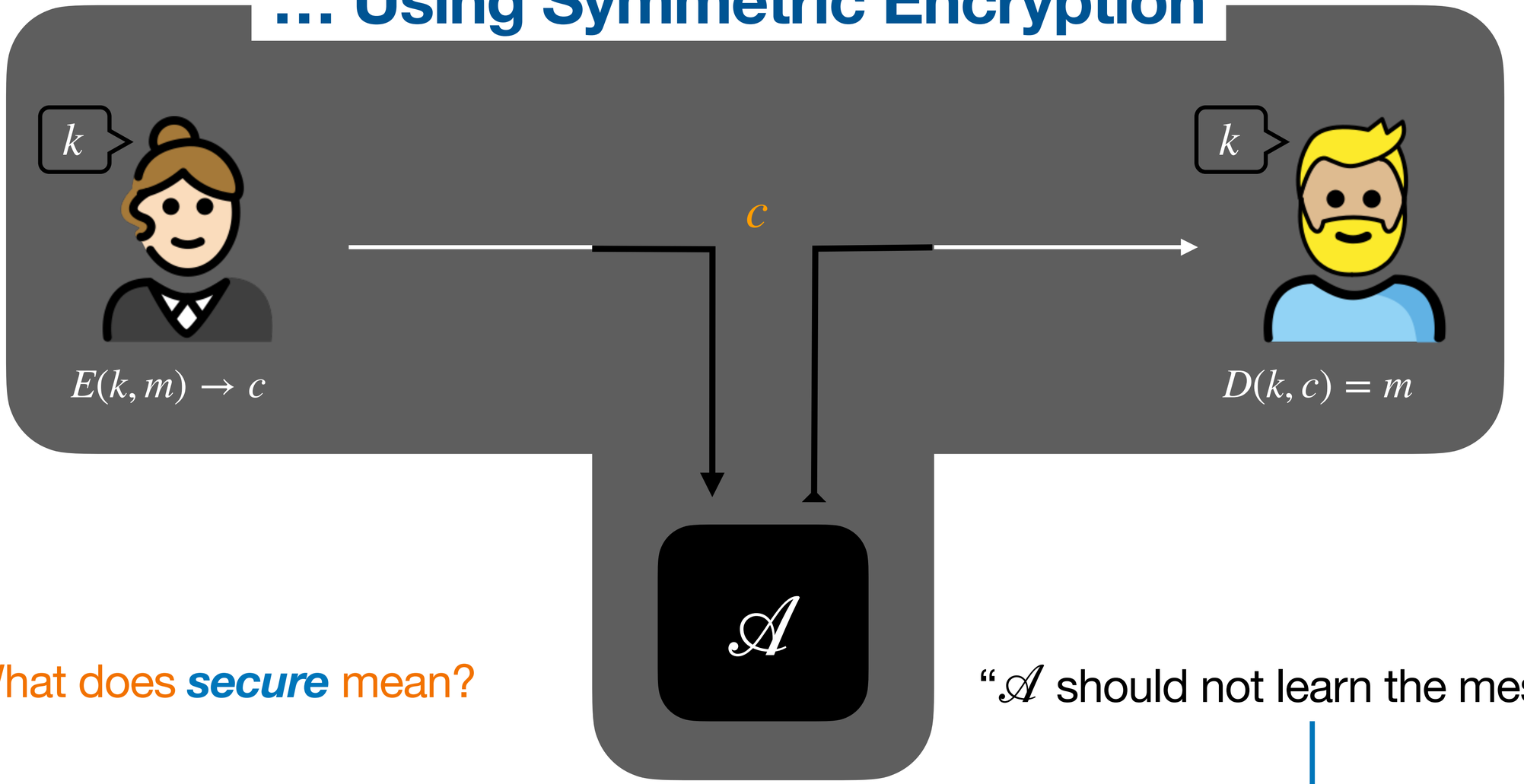
$$k \oplus m = c$$

Uniformly random ciphertext



Secure Communication Over an Insecure Channel

… Using Symmetric Encryption



- 🧐 What does ***secure*** mean?

"$\mathcal{A}$ should not learn the message"

"The ciphertext c should **not leak any information** about the message m"

(a part from the length of m)

Perfect Secrecy

Definition: Perfect Secrecy (Perfect Security)

A symmetric encryption scheme $(KeyGen, E, D)$ is perfectly secret if for all pair of messages $m_0, m_1 \in \mathcal{M}$ and for all ciphertexts c it holds that:

$$Pr[c \leftarrow E(k, m_0)] = Pr[c \leftarrow E(k, m_1)]$$

Where the probabilities are taken over all possible keys $k \leftarrow KeyGen(1^n)$ and randomness in the encryption algorithm, if E is randomised.

This security notions essentially states that:

For every pair of messages $m_0 \neq m_1$, the probabilities that either message maps to a given ciphertext *c* must be equal.

The First Security Proof in This Course

The One-Time Pad is a perfectly secure private-key encryption scheme.

Proof:

First, show that OTP is a symmetric encryption scheme.

To do so, define $\mathcal{K} = \mathcal{M} = \mathcal{C} = \{0,1\}^n$, and the algorithms

$KeyGen(1^n): \ k \leftarrow \$\{0,1\}^n, \quad E(k,m) = k \oplus m,$ and $D(k,c) = k \oplus c.$

Correctness holds since for all k, and all m: $D(k, E(k,m)) = k \oplus c = k \oplus (k \oplus m) = m.$

Next, security. For every message-ciphertex pair (m, c), the only key that maps m to c (i.e., such that $c = E(k,m)$) is $k = m \oplus c$. Then:

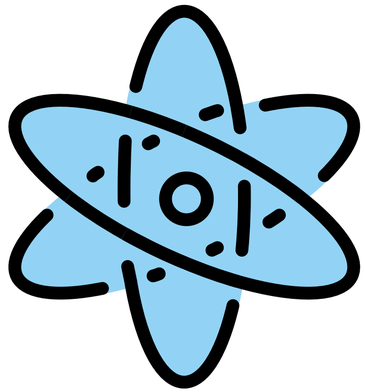
$$\Pr_{k \leftarrow \text{KeyGen}(1^n)}\left[c = E(k,m)\right] = \Pr_{k \leftarrow \text{KeyGen}(1^n)}\left[k = c \oplus m\right] = \frac{1}{|\mathcal{K}|} = 2^{-n}.$$

Since this holds for every message m, we have that for any fix ciphertext c:

$Pr[E(k,m_0) = c] = \frac{1}{|\mathcal{K}|} = Pr[E(k,m_1) = c]$ where the probabilities are take over all possible keys.

Security: Information-Theoretic VS Computational

Perfect-secrecy is a form of **information-theoretic** (aka unconditional) security notion, that means: the security property holds *no matter what* computational power, time-, or space-limit the adversary has.



Computational security is based on the *conjectured* **gap** between the complexity classes P and NP, that means: we have efficient algorithms for the legitimate user, and inefficient algorithm for the adversary.

A **gap** is a hard problem (can be verified quickly but no efficient algorithm is known to solved it). What is *computationally hard* depends on the adversary's resources (classical vs quantum computing, storage, time..)

Computational solutions are usually more efficient and practical than information-theoretic ones **BUT** they always have a "best before date" (we'll discuss this during the course)

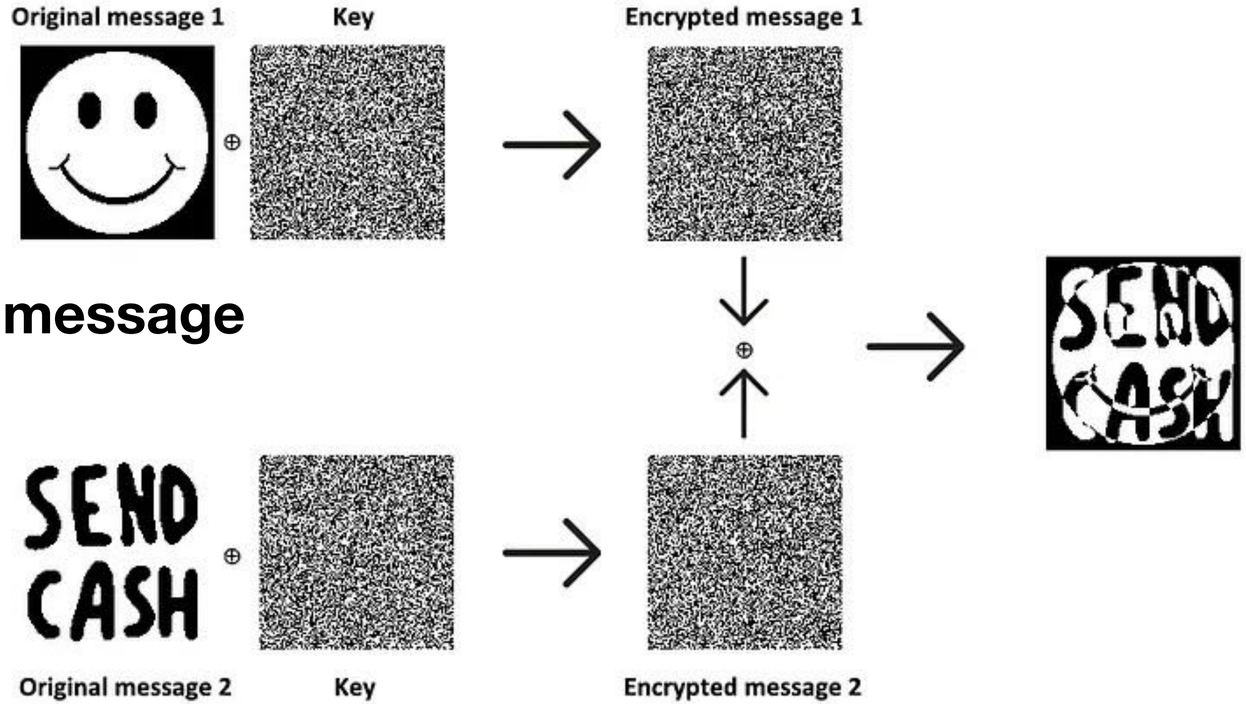
One Time Pad: Problems

1- The key is as long as the message

2- The key should only be used to encrypt ONE message

Assume that the same key is used twice to produce $c_0 = k \oplus m_0$ and $c_1 = k \oplus m_1$.
Assume the adversary $\mathscr{A}$ gets hold of the two ciphertexts, then $\mathscr{A}$ can compute:

$$c_0 \oplus c_1 = (k \oplus m_0) \oplus (k \oplus m_1) = m_0 \oplus m_1$$



Picture taken from https://thusithathilina.medium.com/reused-key-vulnerability-in-one-time-pad-for-ctf-9e1fc04015c

But $m_0 \oplus m_1$ conveys a lot of information about m_0 and m_1. This is not acceptable.

3- The ciphertext is (intentionally!) malleable



Shannon's Theorem

Theorem (Shannon 1940s)
If a symmetric encryption scheme $(KeyGen, E, D)$ defined over $(\mathcal{K}, \mathcal{M}, \mathcal{C})$ has perfect security then $|\mathcal{K}| \geq |\mathcal{M}|$.

Proof: Fix arbitrary $m_0 \in \mathcal{M}$ and $k_0 \in \mathcal{K}$, and let $c_0 = E(k_0, m_0)$.
Since the encryption scheme has perfect secrecy, it holds that "every message has the same probability to be encrypted to *c*", *i.e.*, $Pr[c_0 = E(k, m)] = Pr[c_0 = E(k, m_0)] > 0$.
Thus for each $m_i \in \mathcal{M}$ there is a key $k_i \in \mathcal{K}$ such that $E(k_i, m_i) = c_0$.
But each message must have a different key! If the same key k would map two plaintexts m_1 and m_0 to the same ciphertext $c_0 = E(k, m_1) = E(k, m_0)$, then we lose correctness (the decryption of c_0 for the key k becomes ambiguous).
Thus there must be at least as many keys as messages: $|\mathcal{K}| \geq |\mathcal{M}|$.

Take away: ***perfect security is impractical***


How close to perfect security can we go, while being practical?

A Little Secret: the Core of Crypto Is Randomness



This is the goal of Pseudo Random Generators (PRG)

The perfect secrecy of OTP comes from using one random key to mask/hide one message. Since cannot reuse the key (otherwise we lose security), can we '*expand*' it?

TDA352/DIT352 CRYPTOGRAPHY

Module 1 : Lecture 1 - Agenda

Introduction

- Cryptography: Meaning and Aims
- Brief History
- Kerckhoffs's Principle
- Foundations of Modern Cryptography

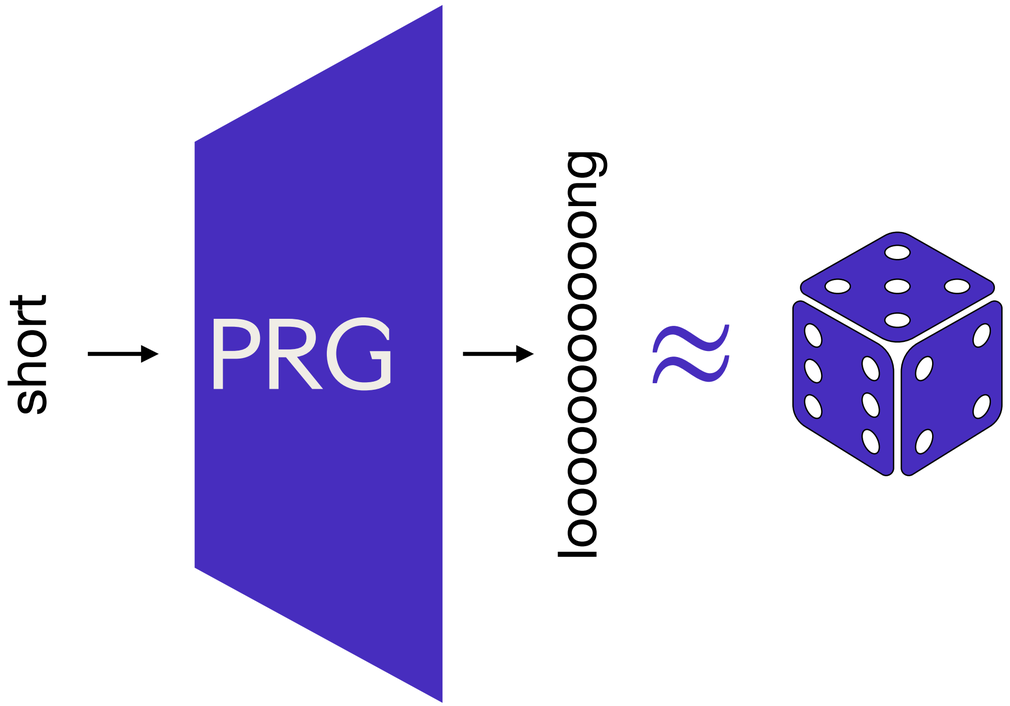
Symmetric Encryption

- Secure Communication
- Symmetric Encryption
- The One Time Pad (OTP) **[Proof]**
- Perfect Secrecy
- Shannon's Theorem **[Proof]**

Pseudorandom Generators (PRG)

- Definition
- Examples

Pseudo Random Generators - an Intuition



A pseudo random generator PRG is a **deterministic**, efficient function that **stretches** a short random input s into a longer output y.

🧐 *security notion?*

For a random input s, n**o efficient algorithm** should be able to tell the output PRG(s) apart from a truly uniform string of the same length, except with **negligible** advantage.

What Does Negligible Mean?

"too small to matter"

Negligible A function $negl(x) : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is **negligible** in x if *for any* positive polynomial $p(x)$ it holds that

$$negl(x) \leq \frac{1}{p(x)} \quad \text{for all } x \geq x_0 \in \mathbb{N}$$

Intuition: Events that occur with negligible probability occur so seldom that polynomial time algorithms will never see them happening.

This definition is asymptotic ("it holds from a certain point onwards"). This is a common approach in complexity-based cryptography.

In practice, if one needs to pick a value: $negl(x) < 2^{-128}$ is considered to be negligible (but this depends on the context, and may yield inefficient constructions).

Pseudo Random Generators (PRG)

A Pseudo Random Generator is a deterministic function PRG : $\{0,1\}^n \rightarrow \{0,1\}^\ell$ such that:

- PRG($\cdot$) is efficiently computable
- PRG($\cdot$) expands its input, i.e., $\ell > n$
- For a random input seed $s \leftarrow \$\{0,1\}^n$ no efficient adversary can tell apart the output of PRG(s) from a random string $k \leftarrow \$\{0,1\}^\ell$.



formal definition in the next lecture

The best way to check if a candidate algorithm is a PRG is by running a series of tests, there is no mathematical proof! But we can reason about the *security* of a PRG using a formal (mathematical) security game.

Example: Not a PRG

Given $s \leftarrow \$\{0,1\}^n$ define PRG : $\{0,1\}^n \rightarrow \{0,1\}^{2n}$ as PRG$(s) = s \,||\, s$

- PRG($\cdot$) is efficiently computable
- PRG($\cdot$) expands its input, i.e., $\ell > n$
- *for* $s \leftarrow \$\{0,1\}^n$ *no efficient adversary can tell apart* PRG(s) *from a random string* $k \leftarrow \$\{0,1\}^{\ell}$.

Example: Sneaky Not a PRG

Given $s \leftarrow \$\{0,1\}^n$ define PRG : $\{0,1\}^n \rightarrow \{0,1\}^{n+1}$ as:

$$\text{PRG}(s) = s \,||\, \oplus_{i=1}^{n} s_i$$

- PRG($\cdot$) is efficiently computable
- PRG($\cdot$) expands its input, i.e., $\ell > n$
- *for* $s \leftarrow \$\{0,1\}^n$ *no efficient adversary can tell apart* PRG(s) *from a random string* $k \leftarrow \$\{0,1\}^{\ell}$.

Example: a PRG

Given $s \leftarrow \$\{0,1\}^n$ define PRG : $\{0,1\}^n \rightarrow \{0,1\}^\ell$ as:

$$\text{PRG}(s) = \sum_{i=1}^{n} s_i \cdot a_i \mod 2^\ell$$ for suitable parameters, this is a secure PRG assuming the subset-sum (knapsack) problem is hard.

for a fixed set of vectors $a_1, \ldots, a_n \leftarrow \$\{0,1,\ldots 2^\ell - 1\}$, with $\ell > n$.

- PRG($\cdot$) is efficiently computable
- PRG($\cdot$) expands its input, i.e., $\ell > n$
- *for* $s \leftarrow \$\{0,1\}^n$ *no efficient adversary can tell apart* PRG(s) *from a random string* $k \leftarrow \$\{0,1\}^\ell$.

for suitable parameters, this is a secure PRG **assuming the subset-sum (knapsack) problem is hard**.

[optional: check out the proof here]